

Department of Telecommunication and Networking Specialized in Cybersecurity

ESP32 Password Manager System

Course : Cryptography Year 3 | Term 1 Instructed by : Mr Meas Sothearath Date of submission : 20/12/2025

Submitted by : Huon Sreynuth

TABLE OF CONTENTS

- 1. Introduction 1.1 Overview of the Project 1.2 Problem Statement 1.3 Motivation 1.4 Related Cryptographic Concepts
 - 2. System Architecture 2.1 Overall Workflow 2.2 Key (Salt) Generation Process 2.3 Password Hashing and Storage Process 2.4 Password Verification Process 2.5 System Flowcharts and Diagrams
 - 3. Implementation Details 3.1 Technologies and Libraries Used 3.2 Project Structure 3.3 Key Functions 3.4 Web UI Implementation 3.5 Security Practices Followed
 - 4. Usage Guide 4.1 Installation and Setup 4.2 Running the Program 4.3 Creating Password Entries 4.4 Verifying Passwords 4.5 Expected Outputs and Examples
 - 5. Conclusion and Future Work 5.1 Summary 5.2 Future Work
 - 6. References
-

1. INTRODUCTION

1.1 Overview of the Project This project implements an ESP32-based password manager that securely stores salted SHA-256 password hashes in Firebase Realtime Database. The ESP32 serves a lightweight web interface that allows users to create and verify password entries. Only salts and hashes are stored; plaintext passwords are never saved.

1.2 Problem Statement Embedded systems and cloud-connected devices are vulnerable if sensitive credentials are stored in plaintext. Without cryptographic protection, attackers can easily compromise passwords. This project addresses the problem by applying hashing with random salts before storage.

1.3 Motivation The motivation of this project is to demonstrate how cryptographic principles can be applied to embedded systems. It helps students understand secure password storage, hashing, random number generation, and cloud database integration using a real IoT device.

1.4 Related Cryptographic Concepts The project applies several cryptographic concepts, including SHA-256 hashing, salting, secure random number generation, and authentication mechanisms. These concepts are essential for ensuring data integrity and security in embedded applications.

1. SYSTEM ARCHITECTURE

2.1 Overall Workflow After booting, the ESP32 connects to WiFi and initializes Firebase. A web server is started to handle user requests. Users interact with the system through a web interface to create or verify password entries. Passwords are salted, hashed, and stored in Firebase, while verification recomputes and compares hashes.

2.2 Key (Salt) Generation Process A random salt is generated using the ESP32 hardware random number generator. The salt is converted to hexadecimal format and combined with the user password before hashing.

2.3 Password Hashing and Storage Process The ESP32 concatenates the generated salt with the password and computes a SHA-256 hash. Both the salt and the resulting hash are stored in Firebase under a unique identifier.

2.4 Password Verification Process During verification, the ESP32 retrieves the stored salt and hash from Firebase, recomputes the hash using the provided password, and compares the result to determine whether the password is valid.

2.5 System Flowcharts and Diagrams The system follows a simple flow consisting of input reception, cryptographic processing, cloud storage access, and result feedback. Flowcharts can be added to visually represent these steps.

1. IMPLEMENTATION DETAILS

3.1 Technologies and Libraries Used The project is developed using Arduino C++ on the ESP32 platform. It utilizes mbedTLS for cryptographic hashing, Firebase ESP Client for cloud communication, and the built-in WebServer library for HTTP services.

3.2 Project Structure The project consists of a single main sketch file for the ESP32, along with documentation and report files. For better maintainability, cryptographic, networking, and web components can be modularized.

3.3 Key Functions The key functions include salt generation, SHA-256 hashing, Firebase read and write operations, WiFi connection handling, and HTTP request processing. Each function performs a specific role in ensuring secure password storage and verification.

3.4 Web UI Implementation The web interface is implemented using static HTML pages embedded in the ESP32 firmware. Forms are provided for creating and verifying password entries. The interface is intentionally minimal to reduce complexity and resource usage.

3.5 Security Practices Followed The system uses hardware-based random number generation for salts and avoids storing plaintext passwords. Sensitive buffers are cleared from memory after use. However, improvements such as stronger key derivation functions and proper authentication mechanisms are recommended.

1. USAGE GUIDE

4.1 Installation and Setup Install the Arduino IDE and ESP32 board support. Add the required Firebase ESP Client library. Configure WiFi credentials and Firebase keys in the source code before uploading.

4.2 Running the Program Upload the sketch to the ESP32 and open the serial monitor to obtain the device IP address. Access the web interface using a browser on the same network.

4.3 Creating Password Entries Users navigate to the create page, enter a name and password, and submit the form. The ESP32 generates a salt, hashes the password, and stores the result in Firebase.

4.4 Verifying Passwords Users navigate to the verify page, enter the required information, and submit. The ESP32 verifies the password by recomputing and comparing hashes.

4.5 Expected Outputs and Examples Successful operations display confirmation messages, while incorrect inputs result in error messages. Firebase stores password entries in a structured JSON format.

1. CONCLUSION AND FUTURE WORK

5.1 Summary The ESP32 Password Manager successfully demonstrates secure password handling on an embedded platform. It integrates cryptographic hashing, random salt generation, and cloud database storage.

5.2 Future Work Future improvements include implementing stronger password derivation algorithms, adding authentication and access control, enabling encrypted communication, and improving code modularity.

1. REFERENCES

Arduino ESP32 Documentation Firebase Realtime Database Documentation mbedTLS Cryptographic Library Documentation NIST Guidelines on Password Storage